ArrayList - Complete Notes

Simple revision notes written in the way we discussed in chat.

This chapter explains ArrayList from zero to interview level with examples, diagrams, complexity, and common mistakes.

1. Why was ArrayList created?

Normal arrays are fast, but they have one big problem: fixed size. If you create an array of size 10 and later need the 11th element, the array cannot grow. ArrayList was created to solve this problem. It behaves like an array that can grow automatically.

Think of seats in a theater. Every seat has a number. If you want seat 50, you go directly to seat 50. That direct access is why array-based structures are fast for reading by index.

Main idea
Array = fixed size
ArrayList = dynamic array
Fast read by index
Resize automatically when full

2. What is ArrayList?

ArrayList is a List implementation in Java. It stores elements in order and allows duplicates. It is used when you need frequent reading by index and appending at the end.

```
Example:

List<Integer> list = new ArrayList<>();
list.add(10);
list.add(20);
list.add(30);

Now list looks like:
[10, 20, 30]
```

ArrayList is part of the List family, so order matters. If you add elements in a certain order, they are stored in that same order.

3. Internal working

Internally ArrayList is backed by an Object[] array. When space is available, a new element is placed at the next free position. When the internal array becomes full, Java creates a bigger array, copies all existing elements into it, and then continues.

```
Conceptual picture:

Before resize:
Index: 0   1   2   3
Data : 10  20  30  40

After adding one more element:
Create bigger array
Copy old elements
Insert the new value
```

Resize is expensive because copying takes time. But it happens only sometimes, not on every add.

4. Why is get(index) so fast?

Because ArrayList uses an array internally. Java can directly jump to a position using the index. It does not need to check previous elements one by one. That is why get(index) is O(1).

Easy memory line
ArrayList = direct jump to index
LinkedList = walk node by node

5. Why is insertion in the middle slow?

Suppose the list is [10, 20, 30, 40]. If you insert 15 at index 1, Java must move 20, 30 and 40 one step right. Because many elements may shift, insertion in the middle is O(n).

```
Before:
[10, 20, 30, 40]

Insert 15 at index 1:
Shift 20, 30, 40 right

After:
[10, 15, 20, 30, 40]
```

6. add(end) and resize

Adding at the end is usually fast because Java places the new element at the next free position. Sometimes the array becomes full, so Java must resize. That resize step is expensive, but since it happens occasionally, the average cost is still good. This is why add(end) is called amortized O(1).

Amortized O(1)
Most add(end) operations are quick
Sometimes resize happens
Average over many operations stays O(1)

7. Index-based operations

Operation	Complexity	Reason
get(index)	O(1)	Direct array access
add(end)	Amortized O(1)	Usually append; resize sometimes
add(index)	O(n)	Shifting elements
remove(index)	O(n)	Shifting elements left
contains()	O(n)	Check one by one

8. Real-life example

Imagine a row of apartment flats. Each flat has a number. If you know the flat number, you can go directly there. That is how ArrayList works for access by index.

```
ArrayList idea:

Index: 0   1   2   3
Data : A   B   C   D

list.get(2) -> C
```

9. ArrayList vs LinkedList

ArrayList	LinkedList
Dynamic array	Doubly linked list
Fast get(index)	Slow get(index)
Middle insert/remove slow	Changing links is faster
Best for reading by index	Best for frequent insert/delete

10. When should you use ArrayList?

Use ArrayList when you need order, duplicates are allowed, and you read data by index often. If you mostly add at the end and read later, ArrayList is usually a good choice.

Good use cases
Student marks list
List of names
Items to display in order
Data where index access matters

11. Common mistakes

A common mistake is thinking ArrayList is slow for everything. That is not true. It is very fast for reading by index. Another mistake is thinking resize happens on every add. Resize happens only when the internal array becomes full.

Some people also think ArrayList and LinkedList are just two names for the same thing. They are completely different internally.

12. Interview questions

What is ArrayList? Why is get(index) O(1)? Why is add(index) O(n)? What does amortized O(1) mean? How is ArrayList different from LinkedList? When would you choose ArrayList?

13. Final revision sheet

Topic	Simple meaning
ArrayList	Dynamic array
Internal storage	Object[] array

get(index)	Fast direct access
add(end)	Usually fast
add(index)	Slow because shifting
remove(index)	Slow because shifting
Best use	Frequent reading by index
Duplicates	Allowed

One-line memory: ArrayList = dynamic array. Fast for reading by index, slow for shifting in the middle.